

Architectural Analysis: Knowledge Management System (KMS)

Project Name: Knowledge Management System (KMS)

Target Application Scope: Final Year Project (FYP) Web Application

Tech Stack: Angular 18+ (Frontend) & Spring Boot 3 / Java 25 (Backend)

Database: Embedded H2 Database with Team-Based RBAC

This report evaluates the KMS architecture for a decoupled Angular/Spring Boot application.

Prerequisite

The secrets in .env file

```
# Database Configuration Profile
H2DB_URL=jdbc:h2:file:./data/kmsdb;DB_CLOSE_ON_EXIT=FALSE
H2DB_USERNAME=meilin
H2DB_PASSWORD=meilin
```

1. Backend RESTful Service Design & API Endpoint Mapping

1.1 Resource-Oriented API Naming Conventions

Endpoint URI	HTTP Method	Business Logic Purpose	Key Parameters
/api/auth/login	POST	Authenticates user credentials & returns user profile with team role.	Body: { username, password }
/api/documents & /document	GET	Retrieves team-filtered, strategy-sorted document list.	Query: sort / sortBy, Header: X-User-Team
/api/documents/{id}	GET	Fetches a single knowledge asset by primary key with team authorization check.	Path: id, Header: X-User-Team
/api/documents	POST	Creates a new knowledge asset and auto-assigns user team.	Body: KnowledgeDocument, Header: X-User-Team
/api/documents/{id}	PUT	Updates an existing asset with team ownership verification.	Path: id, Body: KnowledgeDocument, Header: X-User-Team
/api/documents/{id}	DELETE	Deletes an asset with team ownership verification.	Path: id, Header: X-User-Team

Endpoint URI	HTTP Method	Business Logic Purpose	Key Parameters
/api/documents/search	GET	Filters assets by lifecycle status (Approved, Draft).	Query: status, Header: X-User-Team
/api/documents/popular/{minViews}	GET	Derived JPQL query fetching popular assets with high view counts.	Path: minViews

1.2 Strategy Design Pattern for Backend Document Sorting

Rather than writing rigid database queries or if/else statements for sorting, the system implements the Strategy Design Pattern in package `com.assignment.backend.strategy`. The system utilizes the Strategy Pattern to decouple sorting logic from the controller. Concrete strategies (Title, Status, Tags) are registered in a `DocumentSortContext`, which uses Spring DI to inject a strategy map for dynamic execution based on query parameters.

1.3 Team-Based Access Control (Team RBAC) via Header Interception

Enterprise knowledge assets must be isolated between organizational departments (IT Support, HR, Engineering). By evaluating the custom X-User-Team HTTP header:

- `getAllDocuments` automatically isolates results to `findByTeam(userTeam)`.
- `updateDocument` and `deleteDocument` return `HTTP 403 Forbidden` if a user attempts cross-team mutations.

A `@RestControllerAdvice` handles global exceptions, mapping `ResourceNotFound` and `Validation` errors to structured JSON payloads (404/400/500) to ensure API consistency and security.

2. Frontend Component Architecture & Router Navigation

2.1 Component Hierarchy & Functional Layering

The components in this project:

`LoginComponent` for auth, `DocumentDashboard` as the state container using Angular Signals, and reusable Presenter components like `DocumentRow`. `DocumentForm` handles both create and edit modes via nested routing.

2.2 Navigation Design & Route Guard Protection (app.routes.ts)

- **AuthGuard:** Protects `/dashboard` and `/manage-form`, redirecting unauthenticated users to `/login`.

3. Service Layer Abstraction & HTTP Client Architecture

3.1 Separation of Concerns

Components are responsible for UI presentation and event handling. Network communication and state management are encapsulated in root-level singleton services like `DocumentService` and `AuthService`. This enhances reusability as multiple components consume the same API methods. Also, this ensures maintainability. For instance, URL paths (`/api/documents`), header policies, and HTTP error transformations are centralized in `DocumentService`.

3.2 Header Injection & Angular Signals

`DocumentService` automatically attaches the active team header via `getAuthHeaders()`. `DocumentDashboardComponent` leverages Angular Signals (`documents = signal<Document[]>([])`) to hold state, guaranteeing efficient change detection and reactive UI updates when `loadDocs()` completes.

4. Reactive Forms Design & Validation Strategy

4.1 Hierarchical FormGroup Structure

`DocumentFormComponent` constructs a multi-level reactive form tree using `FormBuilder`.

- **Nested metadata FormGroup:** Groups asset header information separately from payload body.
- **Dynamic tags FormArray:** Allows users to dynamically add (`addTag()`) or remove (`removeTag(i)`) keyword tags.

4.2 Validation Strategy & Validity Gates

Reactive Forms employ a hierarchical `FormGroup` for metadata and a `FormArray` for dynamic tagging. Validation is triggered on 'touched' states, and the UI remains reactive via Angular Signals for efficient state management and change detection.

5. User Experience

5.1 User-Centric Safeguards & Error Resiliency

1. The form provides explicit "Back to Dashboard" and "Cancel" buttons, allowing users to exit form workflows without forced submission.
2. Disabling the "Remove" button on actively edited document rows eliminates frustration caused by editing deleted records.
3. Backend validation errors or forbidden access attempts are intercepted and converted into user-friendly dialogs rather than unhandled console errors.